

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Name: D. Mohit Venkat Sai

Roll No: 2520030106

Section: 8

Practical-4:

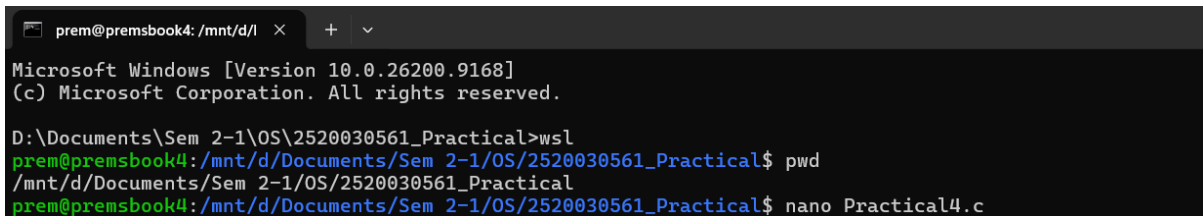
4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Process Synchronization and Zombie Processes

Aim-

To create multiple child processes and synchronize their completion using wait() and waitpid(), and to create, identify, and eliminate a zombie process using proper synchronization.



```
premise@premsbook4: /mnt/d/1 × + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

D:\Documents\Sem 2-1\OS\2520030561_Practical>wsl
premise@premsbook4: /mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ pwd
/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical
premise@premsbook4: /mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ nano Practical4.c
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;

    for(i = 0; i < 3; i++)
    {
        pid = fork();

        if(pid == 0)
        {
            printf("Child %d: PID=%d PPID=%d\n",
                   i+1, getpid(), getppid());
            sleep(2);
            exit(0);
        }

        if(pid < 0)
        {
            perror("fork");
            exit(1);
        }
    }

    for(i = 0; i < 3; i++)
    {
        wait(NULL);
        printf("Parent collected a child\n");
    }

    printf("All children completed\n");

    return 0;
}

```

```

prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ gcc -Wall Practical4.c -o Practical4
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ ./Practical4
Child 1: PID=623 PPID=622
Child 3: PID=625 PPID=622
Child 2: PID=624 PPID=622
Parent collected a child
Parent collected a child
Parent collected a child
All children completed

```

```

prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ nano Practical4_waitpid.c

```

```
#include <stdio.h> Explain and Fix Problem (Ctrl+Shift+.)
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main()
{
    pid_t p1, p2;
    int status;

    p1 = fork();

    if(p1 == 0)
    {
        printf("Child 1: PID=%d\n", getpid());
        sleep(3);
        exit(10);
    }

    p2 = fork();

    if(p2 == 0)
    {
        printf("Child 2: PID=%d\n", getpid());
        sleep(1);
        exit(20);
    }

    waitpid(p2, &status, 0);
    printf("Parent collected Child 2\n");

    waitpid(p1, &status, 0);
    printf("Parent collected Child 1\n");

    printf("All children completed\n");

    return 0;
}
```

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ gcc -Wall Practical4_waitpid.c -o Practical4_waitpid
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ ./Practical4_waitpid
```

```
Child 2: PID=664
Child 1: PID=663
Parent collected Child 2
Parent collected Child 1
All children completed
```

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ nano zombie.c
```

```
#include <stdio.h> Explain and Fix Problem (Ctrl+Shift+.)
#include <stdlib.h>
#include <unistd.h>
```

```
int main()
{
    pid_t pid;

    pid = fork();

    if(pid == 0)
    {
        printf("Child PID=%d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }

    printf("Parent PID=%d\n", getpid());
    printf("Child PID=%d\n", pid);

    printf("Parent sleeping...\n");
    sleep(20);

    return 0;
}
```

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ gcc -Wall zombie.c -o zombie
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ ./zombie
Parent PID=714
Child PID=715
Parent sleeping...
Child PID=715
Child exiting...
```

Open another WSL terminal

While the first program is sleeping, open another terminal.

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
prem 822    0.0    0.0  4128  2388 pts/2    S+   09:20   0:00 grep --color=auto zombie
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ nano zombie_fixed.c
```

```
#include <stdio.h> Explain and Fix Problem (Ctrl+Shift+.)
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if(pid == 0)
    {
        printf("Child PID=%d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }

    printf("Parent PID=%d\n", getpid());
    printf("Child PID=%d\n", pid);

    waitpid(pid, &status, 0);

    printf("Parent collected child\n");
    printf("Zombie eliminated\n");

    return 0;
}
```

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ gcc -Wall zombie_fixed.c -o zombie_fixed
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Practical$ ./zombie_fixed
Parent PID=860
Child PID=861
Child PID=861
Child exiting...
Parent collected child
Zombie eliminated
```